

# Principles of Programming Languages:

## Problem Sheet 1 - First steps

### Question 1

```
1  -- Q1
2  rec foldl(f, x, xs) =
3    if xs = nil then x else foldl(f, f(x, head(xs)), tail(xs));;
4
5  rec foldr(f, x, xs) =
6    if xs = nil then x else f(head(xs), foldr(f, x, tail(xs)));;
7
8  rec append(xs, ys) =
9    if xs = nil then ys else head(xs) : append(tail(xs), ys);;
10
11 val map(f, xs) =
12   let val g(a, b) = append(a, f(b): nil) in
13     foldl(g, nil, xs);;
```

## Question 2

```

1  -- Q2(a)
2  index :: (Eq a) => a -> [a] -> Int
3  -- index x [] = -1
4  -- index x (y:ys)
5  --   | x == y           = 0
6  --   | index x ys == -1 = -1
7  --   | otherwise        = 1 + index x ys
8  index x (y:ys) = index' x (y:ys) 0 where
9    index' :: (Eq a) => a -> [a] -> Int -> Int
10   index' x [] n = -1
11   index' x (y:ys) n
12   | x == y      = n
13   | otherwise   = index' x ys (n + 1)

```

```

1  -- Q2(b)
2  rec index(x, xs, n) =
3    if xs == nil then -1
4    else if x == head(xs) then n
5    else index(x, tail(xs), n+1);;

```

```

1  -- Q2(c)
2  indexc :: (Eq a) => a -> [a] -> Int
3  indexc x y =
4    let z = [i | (i, j) <- zip [0..(length y)] y, j == x] in
5      if null z then -1
6      else head z

```

## Question 3

```

1  Number 3
2  Variable "x"
3  Apply (Variable "+") (Variable "x", Number 3)
4  Apply (Variable "*") [Apply (Variable "+") (Variable "x", Number 3), Number 4]
5  (Let (Val "x" (Number 2)) (Apply (Variable "+") [Variable "x", Number 3]))
6  Let (Val "f" (Lambda ["x"] (Apply (Variable "+") [Variable "x", Number 3]))) (Apply (Variable "f") [Number 2])
7  Apply (Variable "f") [Apply (Variable "g") [Variable "x"]]
8  Apply [Apply (Variable "f") (Variable "g")] [Variable "x"]

```

## Question 4

Yes it is legal, value 5.  $x$  is the argument of the function  $f$ .  $g$  becomes a function that adds its argument to the argument of function  $f$ .

## Question 5

Yes it is legal, value 24.

## Question 6

$f$  is set as a function that adds 3 to its argument, and has nothing to do with  $x$  except when referencing its value at assignment of  $f$ .

## Question 7

An Error: wrong number of args is raised. With too little parameters, the value of unassigned parameters is unknown, and the Error: "param" is not defined can get raised. With too many parameters, although the extra parameters do not affect the result, it can be extremely confusing to use.

It takes the value from the latest argument of the name i.e. value of  $x$  is from the third parameter, not the first. It is sensible, but should be avoided.

## Question 8

Both expressions set the value of  $x = e_1$  in  $e_2$  - the first directly, the second, as an argument in a lambda function.

## Question 9

(a) The expression is essentially syntactic sugar for

$^1 (\text{lambda } (x_1, x_2, \dots, x_n) e)(e_1, e_2, \dots, e_n)$

Replace with the above construct in the parser.

- (b) Replace from the parser the expression with the construct it is syntactic sugar for.
- (c) No idea.

## Question 10

Similar to how round brackets `()` are recognised, with an expression between them, square brackets `[]` can too. When square brackets are used, the expressions in between, separated by commas, can be used as arguments to the `list` primitive under `eval`.